

AI 应用开发知识手册

本手册用于演示文档知识库问答系统（RAG + Agent）。

内容覆盖 RAG 流程、Embedding、Agent

工具调用、LangGraph、记忆系统、安全加固、成本优化、评测与可观测性、MCP。

建议提问方式：直接问某个概念是什么、怎么做、有什么注意事项。

第一章 RAG 的完整流程

RAG 是检索增强生成：先查资料，再让模型基于资料回答，从而减少幻觉。

完整步骤是：文档加载、文本分块、向量化、存入向量库、检索、生成回答。

分块参数包括 `chunk_size` 和 `chunk_overlap`。`chunk_size`

太大会引入无关内容，太小会把一句话切断；`chunk_overlap`

让相邻块保留上下文，避免答案正好落在切分边界上。

检索时用 `top-k` 控制召回数量，用相似度阈值过滤低质量结果。`top-k`

越大召回越全，但噪声也越多；阈值太低会误召回，太高会漏召回。

回答阶段必须要求模型只依据检索到的原文作答，并在末尾给出引用，格式为 [文件名 第X页]。

第二章 Embedding 与向量库选型

Embedding 是把文本转成向量的过程，向量之间的距离表示语义相似度。

TF-IDF 基于关键词权重，不需要下载模型，但只认字面重复，对中文默认分词几乎无效，属于最基础的检索方案。

BGE 是中文语义向量模型，常见选择有 bge-small-zh-v1.5 和 bge-m3，能理解同义表达，适合生产项目。通义 text-embedding-v3 是常用的 API 方案。

向量数据库用于存储向量并支持相似度检索。Chroma 轻量、适合原型；Milvus 适合大规模生产；pgvector 适合已有 PostgreSQL 的团队；Qdrant 适合中小规模生产。

余弦距离越小表示越相似：距离 0 表示几乎一样，距离 1 表示几乎不相关。

第三章 Agent 与工具调用

Agent 的本质是让模型在循环里自己决定调用哪个工具，直到任务完成。

原生 tool calling 的流程是：模型返回 tool_calls（只申请，不执行），程序执行工具，再把结果以 role=tool 的消息回传，并携带 tool_call_id。

tool_call_id 用于把工具结果和模型的调用申请一一配对，缺少它或对不上，模型会重复调用工具甚至报错。

ReAct 使用 Thought、Action、Observation 文本格式，需要解析字符串，稳定性较低；function calling 使用结构化协议，更适合生产环境。

工具出错时要区分类型：业务错误不重试，临时错误（超时、429、5xx）才重试，参数错误和未知工具直接返回错误信息给模型。

第四章 LangGraph 状态编排

LangGraph 用图来编排 Agent：State 是共享数据，Node 是处理步骤，Edge 决定执行顺序。

节点返回的是增量字典，框架会把它合并进 State，因此节点不需要返回完整状态。

条件边根据路由函数返回值决定下一个节点，相当于 if/else 分支。

Checkpoint 保存每一步的 State，配合 thread_id 实现多轮会话记忆和断点恢复。

Human-in-the-loop 使用 interrupt 暂停图，等待人工审批后用 Command(resume=...)

恢复执行，适合删除、发送、支付等危险操作。

第五章 记忆系统设计

短期窗口记忆保存最近几轮对话，直接放进上下文，简单准确但会超长。

摘要记忆把旧对话压缩成一段话，保留用户身份、目标、已做决定和未完成事项，节省 token 但会丢失细节。

结构化档案用 JSON

保存姓名、职业、偏好等稳定事实，读取精准，适合回答“我叫什么”这类问题。

向量检索记忆把历史片段存入向量库，提问时按相似度召回，适合跨会话和大量历史信息。

上下文裁剪应保留 system

消息和最近对话，优先丢弃最旧内容；代价是可能失忆，所以要配合摘要与检索记忆。

第六章 安全加固

提示注入是诱导模型忽略原有指令的攻击方式，分为直接注入和间接注入。

间接注入指文件、网页、工具返回内容里藏有指令，模型可能把它们当命令执行。

防御要分层：提示词层声明规则，代码层做权限控制，输出层做脱敏，流程层对危险操作加人工审批。

工具最小权限要求只暴露必要能力，例如只允许访问 sandbox 目录下的 txt 文件。

路径校验必须先 resolve 再判断是否越界，字符串检查无法防止绕过。

光靠提示词无法完全防御注入，关键底线在代码层的校验与审批。

第七章 成本与延迟优化

缓存：相同问题和模型命中缓存时直接返回，省钱省时间；但个性化问题不能全局缓存。

并行：多个互相独立的请求可以同时发出，总耗时接近最慢的一次；有依赖的 Agent 循环不能并行。

模型路由：简单问题使用便宜快速的小模型，复杂问题使用更强的大模型，可以显著降低成本，但需要评测保证质量不下降。

流式输出不会减少总耗时，但能让用户更早看到内容，改善体验。

第八章 评测与可观测性

评测需要 golden set：一组标准问题和参考答案，用来衡量系统改动前后的效果。

规则校验（关键词、JSON 格式）便宜稳定；LLM-as-judge

能理解语义，但需要评分标准并人工抽查。

回归测试用于比较不同 Prompt 版本：改动前后跑同一套题库，只有指标变好才合并。

可观测性记录每次调用的输入、输出、耗时、token、工具调用和错误，常用 LangSmith 或 Langfuse。

日志与反馈回流把 bad case 导出，再加入评测集，形成发现、修复、回归的闭环。

第九章 MCP 与工具生态

MCP 是 Model Context Protocol，用于标准化 AI 应用连接外部工具和数据。

三个角色：Host 是用户使用的 AI 应用，Client 负责连接，Server 提供工具、资源和提示模板。

Server 提供的三类能力是 Tools、Resources 和 Prompts，其中 Tools 与 function calling 直接对应。

传输方式主要有 stdio 和 Streamable HTTP，本地工具通常使用 stdio。

MCP 让工具可以跨应用复用，但安全上仍要遵守最小权限和人工审批原则。